

Институт перспективных технологий и индустриального программирования
Кафедра индустриального программирования

ОТЧЁТ

по дисциплине «Программирование корпоративных систем»

Практическое задание №2

Тема: «Генератор лабиринтов и поиск пути»
(Тема проекта №20)

Дисциплина: Программирование корпоративных систем
Преподаватель: Сокунов Дмитрий Антонович
Семестр: 2 семестр, 2025–2026 уч. г.
Группа: ЭФБО-03-24
Студент: Казакова Александра Вадимовна

Москва, 2026

СОДЕРЖАНИЕ

1 VISION (ВИДЕНИЕ ПРОЕКТА)	3
1.1 Цели проекта	3
1.2 Целевая аудитория	3
1.3 Основные возможности	4
1.4 Ограничения	4
1.5 Отличия от конкурентов	4
2 SRS — SOFTWARE REQUIREMENTS SPECIFICATION	5
2.1 Введение	5
2.2 Общее описание продукта	5
2.3 Методология сбора требований	6
2.4 Требования	6
2.4.1 Бизнес-требования	6
2.4.2 Пользовательские требования	7
2.4.3 Функциональные требования	7
2.4.4 Нефункциональные требования	8
2.5 UML-диаграммы	9
2.5.1 Диаграмма вариантов использования (Use Case Diagram)	9
2.5.2 Диаграмма классов (Class Diagram)	10
2.5.3 Диаграмма последовательности (Sequence Diagram)	11
3 BACKLOG ПРОЕКТА	19
4 ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ	21

1 VISION — ВИДЕНИЕ ПРОЕКТА

Настоящий документ определяет концепцию проекта «Генератор лабиринтов и поиск пути» — консольного приложения, созданного для автоматического построения случайных лабиринтов и отыскания кратчайших маршрутов между заданными точками. Программный продукт разрабатывается на языке C++.

1.1 Цели проекта

- 1) разработать механизмы генерации лабиринтов, базирующиеся на алгоритмах Depth-First Search (DFS) и Kruskal;
- 2) реализовать поиск оптимальных траекторий движения с применением алгоритмов A* и BFS;
- 3) обеспечить наглядную демонстрацию структуры лабиринта и построенного маршрута в консольном режиме;
- 4) спроектировать модульную архитектуру, допускающую подключение дополнительных алгоритмов построения и маршрутизации;
- 5) предоставить инструментарий для сопоставления эффективности различных алгоритмических подходов.

1.2 Целевая аудитория

Сегмент ЦА	Потребность
Обучающиеся технических специальностей	Освоение алгоритмов работы с графами, направлений структур данных (DSU, стеки, очереди, приоритетные очереди) и методов поиска маршрутов.
Преподавательский состав	Использование в образовательном процессе для иллюстрации работы алгоритмов на практических занятиях.
IT-специалисты	Изучение особенностей реализации A*, BFS, соискатели методов генерации лабиринтов, подготовка к техническим собеседованиям.

Сегмент ЦА	Потребность
Учащиеся старших классов	Первичное погружение в алгоритмику на общеобразовательных графах и эвристические методы поиска учреждений.

1.3 Основные возможности

- 1) построение лабиринтов двумя способами: DFS (рекурсивный обход с возвратом) и Kruskal (с применением системы непересекающихся множеств);
- 2) вывод лабиринта на экран с использованием псевдографических символов (стены, проходы, точки старта и финиша);
- 3) прокладывание кратчайшего маршрута между стартовой и финишной позициями с помощью BFS (обход в ширину) и A* (эвристический алгоритм с манхэттенской метрикой);
- 4) визуализация построенного маршрута поверх карты лабиринта;
- 5) конфигурирование габаритов лабиринта (количество ячеек по горизонтали и вертикали);
- 6) вывод измерительных показателей: продолжительность генерации и поиска, количество обследованных ячеек, протяжённость найденного пути;
- 7) интерактивное меню для выбора алгоритмических стратегий.

1.4 Ограничения

- 1) реализация выполняется на C++ с использованием исключительно стандартной библиотеки (STL), без привлечения внешних графических фреймворков;
- 2) структура лабиринта представляет собой прямоугольную сетку; максимальный размер ограничен возможностями консольного вывода (рекомендованный предел — 50×50 ячеек);
- 3) визуализация осуществляется только в текстовом формате (консоль), графическая оболочка отсутствует;

-
- 4) программный продукт функционирует как настольное приложение под управлением Windows и Linux;
 - 5) формируемый лабиринт относится к категории идеальных (не содержит циклов, между любыми двумя точками существует единственный путь);
 - 6) точка отправления располагается в левом верхнем углу (0,0), точка назначения — в правом нижнем углу (height-1, width-1).

1.5 Отличия от конкурентов

Критерий	Конкуренты (веб-сервисы)	Наш проект
Технологический	JavaScript / браузер	C++ — нативная производительность
Вариативность	Только отдельные алгоритмы	Параллельный запуск нескольких
Поиск маршрута	Требуется интернет	Полностью офлайн
Режим работы	Ограниченная	Полная (ручной + генератор)
Формат	Нет	CSV со статистикой
Аналитический	Базовая	Многоуровневая (сравнение/обмен/пивот)
Модифицируемость	Закрывающийся код	Открытый код, простота добавления новых алгоритмов

2.1 Введение

Цель документа

Настоящая спецификация задаёт совокупность требований к программному обеспечению (SRS) для проекта «Генератор лабиринтов и поиск пути». Документ ориентирован на разработчиков, тестировщиков и заинтересованных лиц. В нём изложены функциональные и нефункциональные требования, архитектурные ограничения и сценарии взаимодействия.

Область действия (Scope)

Продукт представляет собой консольное приложение на C++, обеспечивающее генерацию лабиринтов посредством алгоритмов DFS и Kruskal, а также поиск маршрутов с использованием алгоритмов BFS и A*. Система функционирует автономно, не требует сетевых взаимодействий и не имеет графического интерфейса.

Определения и аббревиатуры

Термин / Аббревиатура	Определение
SRS	Software Requirements Specification — спецификация требований к программному продукту
ЦА	Целевая аудитория — совокупность пользователей, на которых ориентирована система
CLI	Command Line Interface — интерфейс командной строки
DFS	Depth-First Search — алгоритм обхода графа в глубину
BFS	Breadth-First Search — алгоритм обхода графа в ширину

Термин / Аббревиатура	Определение
A*	Алгоритм поиска оптимального пути с применением эвристической оценки
DSU	Disjoint Set Union — структура данных для управления непересекающимися множествами
STL	Standard Template Library — стандартная библиотека шаблонов C++
UML	Unified Modeling Language — унифицированный язык графического моделирования
Use Case	Вариант использования — описание взаимодействия актора с системой

2.2 Общее описание продукта

Описание продукта

«Генератор лабиринтов и поиск пути» — это приложение командной строки, написанное на C++, которое создаёт случайные лабиринты и определяет в них кратчайшие маршруты. Программа отображает лабиринт с помощью условных обозначений (# — стена, пробел — проход, S — начало, F — конец, . — проложенный путь) и предоставляет пользователю возможность выбора алгоритмов через текстовое меню.

Функции продукта

- 1) выбор стратегии генерации лабиринта (DFS либо Kruskal);
- 2) выбор метода поиска маршрута (BFS либо A*);
- 3) задание габаритов лабиринта (количество ячеек по горизонтали и вертикали);
- 4) построение случайного лабиринта с применением выбранного алгоритма;
- 5) прокладывание маршрута от стартовой до финишной позиции;
- 6) отображение лабиринта и построенного маршрута в консоли;
- 7) вывод измерительной информации о работе алгоритмов;
- 8) возможность повторного формирования лабиринта и поиска.

Характеристики пользователей

Основную аудиторию составляют студенты и преподаватели, изучающие графовые алгоритмы. Для работы с приложением не требуется навыков программирования — взаимодействие осуществляется через интуитивно понятное текстовое меню.

2.3 Методология сбора требований

Сбор требований выполнялся с использованием комбинированной методики, включающей:

- 1) **анализ предметной области** — изучение существующих реализаций генераторов лабиринтов и алгоритмов поиска маршрутов с целью выявления преимуществ и недостатков;
- 2) **User Stories (Agile)** — формулировка потребностей в виде пользовательских сценариев («Будучи студентом, я хочу наблюдать за работой A*, чтобы разобраться в принципах эвристического поиска»);
- 3) **Story Mapping** — построение карты пользовательских историй для расстановки приоритетов и распределения функциональности по итерациям;
- 4) **анализ источников** — изучение учебной литературы по графовым алгоритмам для формирования исчерпывающего перечня функций.

Все требования структурированы иерархически в соответствии с классификацией Вигерса: бизнес-требования → пользовательские требования → функциональные требования → нефункциональные требования.

2.4 Требования

2.4.1 Бизнес-требования

Бизнес-требования отражают цели и выгоды, которые система должна обеспечить заказчику или организации.

ID	Приоритет	Требование
BR-01	Высокий	Система должна выступать в роли эффективного учебного инструмента, демонстрирующего работу графовых алгоритмов.
BR-02	Высокий	Приложение должно быть готово к применению в учебном процессе в течение одного семестра разработки.
BR-03	Средний	Продукт должен функционировать автономно без доступа к сети Интернет.
BR-04	Средний	Система должна подходить для самостоятельного освоения алгоритмов генерации лабиринтов и поиска маршрутов.
BR-05	Низкий	Обеспечение возможности сопоставления производительности различных алгоритмов для последующего анализа.

2.4.2 Пользовательские требования

Пользовательские требования описывают задачи, которые пользователь должен иметь возможность решать с помощью системы.

ID	Приоритет	Требование
UR-01	Высокий	Как пользователь, я хочу выбирать способ генерации лабиринта (DFS или Kruskal), чтобы изучать их отличительные черты.
UR-02	Высокий	Как пользователь, я хочу выбирать метод поиска маршрута (BFS или A*), чтобы сравнивать их эффективность.
UR-03	Высокий	Как пользователь, я хочу визуально оценивать структуру лабиринта, чтобы понимать его устройство.
UR-04	Высокий	Как пользователь, я хочу наблюдать проложенный маршрут на карте лабиринта, чтобы оценивать его протяжённость.
UR-05	Средний	Как пользователь, я хочу указывать размеры лабиринта, чтобы создавать структуры разной сложности.
UR-06	Средний	Как пользователь, я хочу знакомиться со статистическими данными о работе алгоритмов,

ID	Приоритет	Требование
		чтобы проводить сравнительный анализ их производительности.
UR-07	Средний	Как пользователь, я хочу формировать новый лабиринт без перезапуска программы, чтобы тестировать различные конфигурации.
UR-08	Низкий	Как пользователь, я хочу различать стены, проходы, старт, финиш и маршрут с помощью различных символов.

2.4.3 Функциональные требования

Функциональные требования описывают конкретное поведение системы.

ID	Приоритет	Требование
FR-01	Высокий	Система должна обеспечивать генерацию лабиринта с использованием алгоритма Depth-First Search (рекурсивный обход с возвратом).
FR-02	Высокий	Система должна обеспечивать генерацию лабиринта с использованием алгоритма Kruskal (с применением DSU).
FR-03	Высокий	Система должна обеспечивать поиск маршрута с использованием алгоритма BFS (обход в ширину).
FR-04	Высокий	Система должна обеспечивать поиск маршрута с использованием алгоритма A* с манхэттенской эвристикой.
FR-05	Высокий	Система должна отображать лабиринт в консоли, применяя символы: '#' для стен, пробел для проходов.
FR-06	Высокий	Система должна обозначать стартовую позицию как 'S', финишную как 'F', а построенный маршрут как '..'.
FR-07	Высокий	Система должна предоставлять выбор алгоритмов и настройку параметров посредством текстового меню.
FR-08	Средний	Система должна измерять и выводить продолжительность генерации лабиринта в миллисекундах.

ID	Приоритет	Требование
FR-09	Средний	Система должна отображать количество ячеек, обследованных в процессе поиска маршрута.
FR-10	Средний	Система должна отображать длину найденного маршрута в ячейках.
FR-11	Низкий	Система должна проверять существование маршрута между стартом и финишем для расширяемости системы (на случай неидеальных лабиринтов).
FR-12	Низкий	Система должна поддерживать возможность повторной генерации с сохранением заданных параметров.

2.4.4 Нефункциональные требования

ID	Категория	Требование
NFR-01	Производительность	Формирование лабиринта размером 50×50 должно занимать не более 1 секунды.
NFR-02	Производительность	Поиск маршрута в лабиринте 50×50 не должен превышать 0.5 секунды.
NFR-03	Производительность	Время запуска приложения должно составлять не более 1 секунды.
NFR-04	Надёжность	Приложение не должно аварийно завершаться при некорректном вводе размеров; необходима обработка ошибочных ситуаций.
NFR-05	Удобство	Меню должно быть понятным для пользователя использования без предварительной подготовки.
NFR-06	Портативность	Приложение должно компилироваться и корректно функционировать на Windows 10+ и Linux (Ubuntu 20.04+) с использованием компиляторов GCC, Clang или MSVC.
NFR-07	Сопровождаемость	Исходный код должен соответствовать стандарту Google C++ Style Guide; покрытие модульными тестами — не менее 50% алгоритмической составляющей.
NFR-08	Ресурсоёмкость	Потребление оперативной памяти не должно превышать 100 МБ при работе с лабиринтом максимального размера 50×50.

2.5 UML-диаграммы

2.5.1 Диаграмма вариантов использования (Use Case Diagram)

Диаграмма отражает функциональные возможности системы и акторов, взаимодействующих с ней. Выделены два актора: Пользователь (основной) и Файловая система (внешний актор для экспорта).

Примечание: диаграмма строится в draw.io на основе таблицы ролей ниже.

Актор	Вариант использования
Пользователь	Сформировать лабиринт
Пользователь	Выбрать алгоритм генерации (include)
Пользователь	Задать размеры лабиринта
Пользователь	Проложить маршрут
Пользователь	Выбрать метод поиска пути (include)
Пользователь	Создать лабиринт заново
Пользователь	Просмотреть статистику выполнения
Пользователь	Настроить параметры отображения
Пользователь	Экспортировать результаты
Файловая система	Сохранить результаты (экспорт)



Рисунок 2.5.1 - Use Case Diagram

2.5.2 Диаграмма классов (Class Diagram)

Диаграмма классов отражает статическую архитектуру системы: основные сущности, их атрибуты, методы и отношения.

Maze — главный класс, представляющий лабиринт
Атрибуты: - width: int - height: int - grid: vector<vector<Cell>> - startX: int - startY: int - finishX: int - finishY: int
Методы: + Maze(width, height) + getCell(x, y): Cell& + setCell(x, y, type) + display(): void + reset(): void + isValidCell(x, y): bool

Cell — структура, описывающая отдельную ячейку лабиринта
Атрибуты: - walls: struct { top, right, bottom, left: bool } - visited: bool - type: CellType (WALL, PATH, START, FINISH, ROUTE) - row: int - col: int
Методы: + hasWall(direction): bool + removeWall(direction): void + setType(type): void + getType(): CellType

IMazeGenerator — интерфейс (абстрактный класс) генератора лабиринта
Методы: + generate(width, height): Maze* + getName(): string + reset(): void

DFSGenerator — реализует IMazeGenerator
Атрибуты: - stack: vector<Cell> - random: mt19937
Методы: + generate(): Maze* - recursiveBacktrack(x, y): void - getRandomUnvisitedNeighbor(x, y): Cell*

KruskalGenerator — реализует IMazeGenerator (алгоритм Крускала)
Атрибуты: - walls: vector<Wall> - dsu: DisjointSetUnion - random: mt19937
Методы: + generate(): Maze* - initWalls(): void - shuffleWalls(): void

DisjointSetUnion — структура данных для системы непересекающихся
Атрибуты: - parent: vector<int> - rank: vector<int>
Методы: + DSU(n) + find(x): int + unite(x, y): bool

IPathFinder — интерфейс (абстрактный класс) поиска пути
Методы: + findPath(maze: Maze&): vector<Point> + getName(): string + getStats(): PathStats

BFSFinder — реализует IPathFinder (алгоритм поиска в ширину)
Атрибуты: - visited: vector<vector<bool>>

- parent: vector<vector<Point>>
- stats: PathStats
- queue: queue<Point>

Методы:

- + findPath(): vector<Point>
- reconstructPath(): vector<Point>

AStarFinder — реализует IPathFinder (A* с эвристикой Манхэттена)

Атрибуты:

- openSet: priority_queue<Node>
- gScore: vector<vector<int>>
- fScore: vector<vector<int>>
- parent: vector<vector<Point>>
- stats: PathStats

Методы:

- + findPath(): vector<Point>
- heuristic(x1, y1, x2, y2): int (Манхэттен)
- reconstructPath(): vector<Point>

PathStats — структура статистики поиска пути

Атрибуты:

- generationTimeMs: double
- searchTimeMs: double
- visitedCells: int
- pathLength: int
- algorithmName: string

Методы:

- + display(): void
- + exportCSV(): string

Wall — структура, представляющая стену между ячейками

Атрибуты:

- x: int
- y: int
- direction: Direction (HORIZONTAL, VERTICAL)

Point — структура, представляющая координаты

Атрибуты:

- x: int
- y: int

Методы:

+ operator==(other): bool
+ operator!= (other): bool

ConsoleUI — отвечает за взаимодействие с пользователем

Атрибуты:

- maze: Maze*
- generator: IMazeGenerator*
- finder: IPathFinder*
- running: bool

Методы:

+ showMenu(): void
+ getMazeSize(): pair<int,int>
+ selectGenerator(): IMazeGenerator*
+ selectFinder(): IPathFinder*
+ displayMaze(): void
+ displayStats(stats): void
+ run(): void

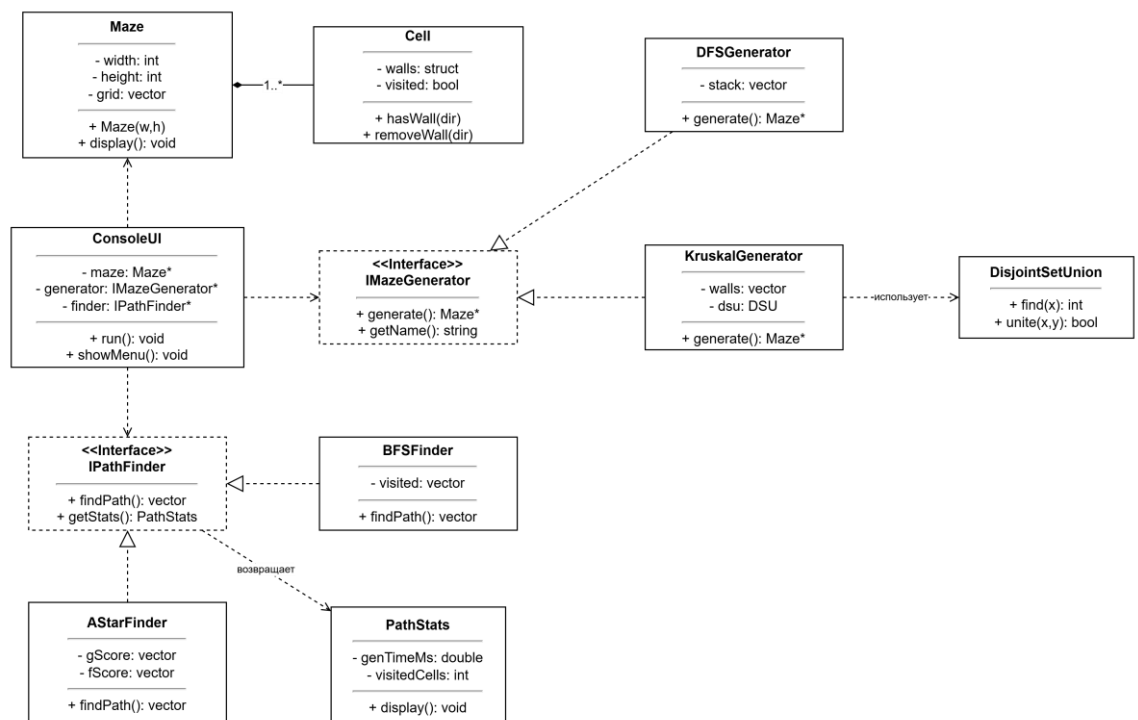


Рисунок 2.5.2 - Class Diagram

Отношения между классами включают наследование, композицию и ассоциацию. По наследованию DFSGenerator и KruskalGenerator являются наследниками IMazeGenerator, а BFSFinder и AStarFinder наследуют IPathFinder. Композиция выражена в том, что Maze содержит множество объектов Cell (1..*). Ассоциативные связи: ConsoleUI использует Maze, IMazeGenerator и IPathFinder; KruskalGenerator использует DisjointSetUnion и Wall; IPathFinder возвращает PathStats и использует Point; IMazeGenerator создаёт Maze, а IPathFinder анализирует его для поиска пути.

2.5.3 Диаграмма последовательности (Sequence Diagram)

Диаграмма описывает сценарий «Генерация лабиринта и поиск пути»: взаимодействие объектов при выборе алгоритма, генерации лабиринта и поиске пути.

№	От	К	Сообщение / Действие
1	Пользователь	ConsoleUI	selectGenerator(DFS)
2	ConsoleUI	IMazeGenerator	new DFSGenerator()
3	Пользователь	ConsoleUI	setMazeSize(21, 21)
4	Пользователь	ConsoleUI	generateMaze()
5	ConsoleUI	IMazeGenerator	generate(width, height)
6	IMazeGenerator	Maze	создаёт структуру лабиринта
7	IMazeGenerator	Maze	recursiveBacktrack(0,0) — убирает стены
8	IMazeGenerator	ConsoleUI	возвращает Maze*
9	ConsoleUI	Maze	display() — отображает лабиринт
10	Пользователь	ConsoleUI	selectFinder(AStar)
11	ConsoleUI	IPathFinder	new AStarFinder()
12	Пользователь	ConsoleUI	findPath()
13	ConsoleUI	IPathFinder	findPath(maze, start, finish)
14	IPathFinder	Maze	getCell(x,y) — проверка проходимости
15	IPathFinder	IPathFinder	эвристика Манхэттена, обход в приоритетной очереди
16	IPathFinder	ConsoleUI	возвращает vector<Point> (путь)
17	ConsoleUI	Maze	отображает лабиринт с путём

№	От	К	Сообщение / Действие
18	ConsoleUI	ConsoleUI	displayStats(время, посещённые ячейки, длина пути)
19	Пользователь	ConsoleUI	exportStats()
20	ConsoleUI	PathStats	exportCSV()
21	PathStats	Файловая система	запись в файл stats.csv

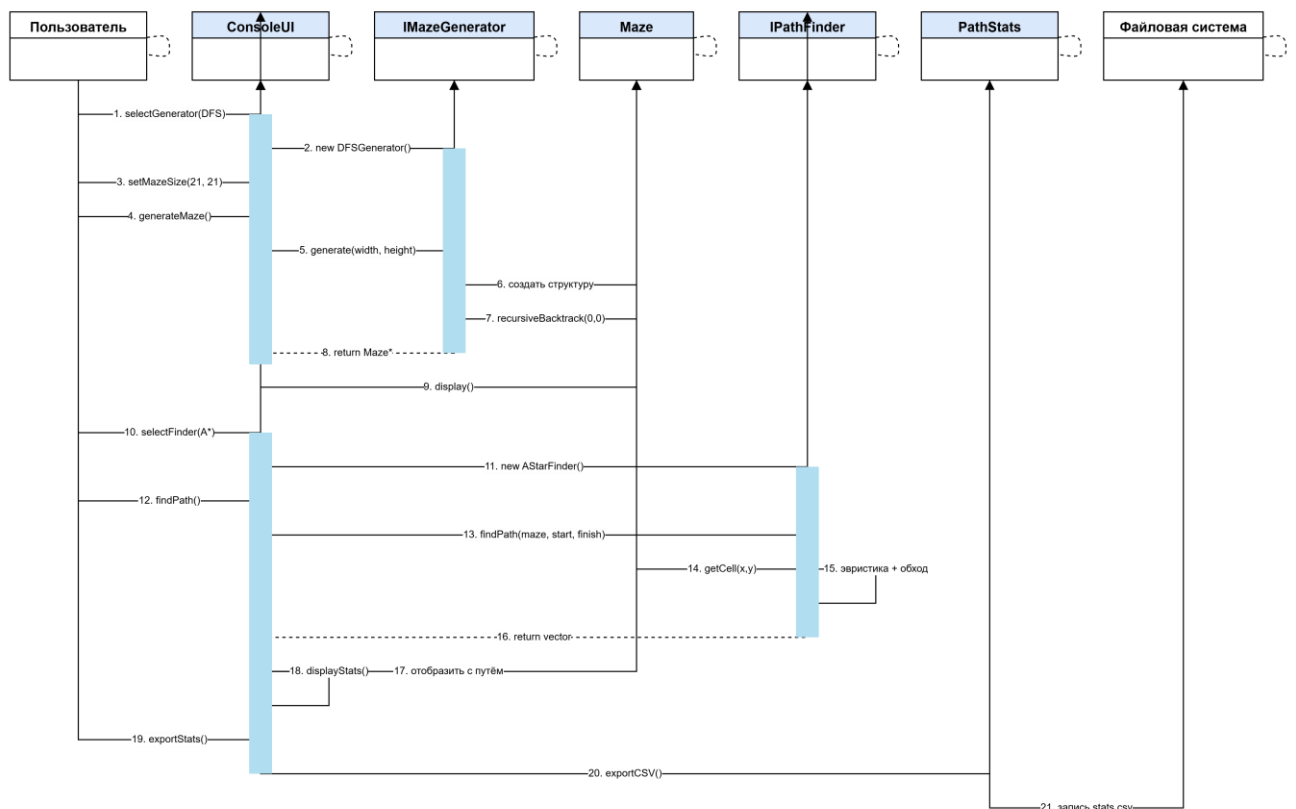


Рисунок 2.5.3 - Sequence Diagram

3 BACKLOG ПРОЕКТА

Backlog представляет собой упорядоченный по приоритету перечень задач для реализации проекта. Приоритеты определены методом MoSCoW (Must have, Should have, Could have, Won't have). Оценка трудоёмкости в Story Points (SP) по шкале Фибоначчи.

3.1 Product Backlog

ID	Задача	MoSCoW	SP	Итерация
BL-01	Разработать структуру Cell и класс Maze с базовыми операциями	Must	3	1
BL-02	Реализовать алгоритм DFS для построения лабиринта (рекурсивный обход)	Must	5	1
BL-03	Реализовать алгоритм Kruskal для построения лабиринта (с DSU)	Must	5	1
BL-04	Создать абстрактный класс IMazeGenerator	Must	2	1
BL-05	Реализовать вывод лабиринта в консоль (символы # и пробел)	Must	3	1
BL-06	Добавить отображение старта (S) и финиша (F)	Must	1	1
BL-07	Создать текстовое меню выбора алгоритма генерации	Must	3	1
BL-08	Реализовать ввод размеров лабиринта с валидацией	Must	2	1
BL-09	Реализовать алгоритм BFS для поиска маршрута	Must	5	2
BL-10	Создать абстрактный класс IPathFinder	Must	2	2
BL-11	Реализовать алгоритм A* для поиска маршрута с манхэттенской эвристикой	Should	8	2
BL-12	Реализовать отображение найденного маршрута (символ .)	Should	3	2
BL-13	Добавить измеритель времени выполнения генерации	Should	2	2
BL-14	Добавить измеритель времени выполнения поиска	Should	2	2

ID	Задача	MoSCoW	SP	Итерация
BL-15	Добавить отображение количества обследованных ячеек	Should	2	2
BL-16	Добавить отображение длины маршрута в ячейках	Should	1	2
BL-17	Добавить возможность повторной генерации без перезапуска	Should	2	2
BL-18	Реализовать проверку корректности размеров (нечётные значения)	Could	2	3
BL-19	Добавить цветовое выделение в консоли (поддержка ANSI)	Could	3	3
BL-20	Добавить сохранение лабиринта в текстовый файл	Could	3	3
BL-21	Добавить загрузку лабиринта из файла	Could	3	3
BL-22	Режим одновременного сравнения двух алгоритмов поиска	Could	5	3
BL-23	Графический интерфейс (SDL2/Qt/ImGui)	Won't	—	—
BL-24	Веб-версия (WebAssembly)	Won't	—	—

Легенда MoSCoW:

Must have — обязательно для минимально жизнеспособного продукта (MVP);

Should have — желательно включить;

Could have — при наличии времени;

Won't have — не планируется в текущем релизе.

3.2 Sprint-план (предварительный)

Итерация	Цель	Задачи	SP (итого)
Sprint 1 (2 нед.)	MVP: генерация лабиринтов и базовая визуализация	BL-01 — BL-08	24 SP
Sprint 2 (2 нед.)	Поиск маршрутов + статистика	BL-09 — BL-17	25 SP
Sprint 3 (2 нед.)	Расширенный функционал и оптимизация	BL-18 — BL-22	16 SP

4 ПАТТЕРНЫ ПРОЕКТИРОВАНИЯ

4.1 Выбор паттернов для проекта

В рамках курса были изучены порождающие, структурные и поведенческие паттерны. Применительно к проекту «Генератор лабиринтов и поиск пути» отобраны три паттерна, соответствующих архитектуре системы.

Таблица 4.1 — Отобранные паттерны

Паттерн	Категория	Назначение в проекте
Стратегия	Поведенческий	Взаимозаменяемость алгоритмов генерации лабиринта и поиска пути
Фабричный метод	Порождающий	Создание объектов-генераторов и объектов-поисковиков без привязки к конкретным классам
Фасад	Структурный	Единый интерфейс для упрощённого управления генерацией, поиском и отображением

4.2 Паттерн «Стратегия»

Интерфейсы IMazeGenerator и IPathFinder из диаграммы классов (раздел 2.5.2) реализуют паттерн «Стратегия». ConsoleUI хранит указатель на текущий алгоритм и может подменять его во время выполнения.

Участники:

- IMazeGenerator — **интерфейс стратегии генерации**
- DFSGenerator, KruskalGenerator — **конкретные стратегии генерации**

-
- IPathFinder — **интерфейс стратегии поиска пути**
 - BFSFinder, AStarFinder — **конкретные стратегии поиска**

Пример реализации:

```
class IMazeGenerator {
public:
    virtual Maze* generate(int width, int height) = 0;
    virtual ~IMazeGenerator() = default;
};
```

```
class DFSGenerator : public IMazeGenerator {
public:
    Maze* generate(int width, int height) override {
        Maze* maze = new Maze(width, height);
        recursiveBacktrack(maze, 0, 0);
        return maze;
    }
};
```

```
class KruskalGenerator : public IMazeGenerator {
public:
    Maze* generate(int width, int height) override {
        Maze* maze = new Maze(width, height);
        processWalls(maze);
        return maze;
    }
};
```

Использование в ConsoleUI:

```
class ConsoleUI {
    IMazeGenerator* generator;
public:
    void setGenerator(IMazeGenerator* g) { generator = g; }
    void generateMaze() { maze = generator->generate(width, height); }
};
```

4.3 Паттерн «Фабричный метод»

ConsoleUI не создаёт генераторы и поисковики напрямую через new, а обращается к классам-фабрикам. Это позволяет добавлять новые алгоритмы, не меняя код меню.

Участники:

- MazeGeneratorFactory — абстрактная фабрика генераторов
- DFSGeneratorFactory, KruskalGeneratorFactory — конкретные фабрики
- PathfinderFactory — абстрактная фабрика поисковиков
- BFSFinderFactory, AStarFinderFactory — конкретные фабрики

Пример реализации:

```
class MazeGeneratorFactory {
public:
    virtual IMazeGenerator* createGenerator() = 0;
    virtual ~MazeGeneratorFactory() = default;
};

class DFSGeneratorFactory : public MazeGeneratorFactory {
public:
    IMazeGenerator* createGenerator() override {
        return new DFSGenerator();
    }
};

class KruskalGeneratorFactory : public MazeGeneratorFactory {
public:
    IMazeGenerator* createGenerator() override {
        return new KruskalGenerator();
    }
};
```

Использование:

```
MazeGeneratorFactory* factory;
if (choice == 1)
    factory = new DFSGeneratorFactory();
else
    factory = new KruskalGeneratorFactory();

IMazeGenerator* gen = factory->createGenerator();
```

4.4 Паттерн «Фасад»

Вместо того чтобы в main.cpp работать с генератором, поисковиком и отрисовщиком по отдельности, создаётся класс MazeFacade, объединяющий все операции за простым интерфейсом.

Участники:

- MazeFacade — фасад, скрывающий внутреннюю сложность
- Maze, IMazeGenerator, IPathFinder, ConsoleUI — компоненты подсистемы

Пример реализации:

```
class MazeFacade {
    IMazeGenerator* generator;
    IPathFinder* finder;
    Maze* maze;
public:
    void setGenerator(IMazeGenerator* g) { generator = g; }
    void setFinder(IPathFinder* f) { finder = f; }

    void generate(int w, int h) {
        maze = generator->generate(w, h);
    }

    void findPath() {
        Point start{0, 0};
        Point finish{maze->getHeight() - 1, maze->getWidth() - 1};
        auto path = finder->findPath(*maze, start, finish);
        maze->applyPath(path);
    }

    void display() { maze->display(); }
};
```

Использование в main.cpp:

```
int main() {
    MazeFacade facade;
    facade.setGenerator(new DFSGenerator());
    facade.generate(21, 21);
    facade.display();
    facade.setFinder(new AStarFinder());
    facade.findPath();
    facade.display();
    return 0;
}
```

4.5 Сводная таблица

Паттерн	Классы проекта
Стратегия	IMazeGenerator → DFSGenerator, KruskalGenerator; IPathFinder → BFSFinder, AStarFinder
Фабричный метод	MazeGeneratorFactory → DFSGeneratorFactory, KruskalGeneratorFactory; PathFinderFactory → BFSFinderFactory, AStarFinderFactory
Фасад	MazeFacade объединяет Maze, IMazeGenerator, IPathFinder, ConsoleUI